

Code review — mini-proiecte multithreading

Runda 1 · 2026-09-29 · sha d845a33 · Alex

Runda 1 — 2026-09-29 — sha d845a33

Ai livrat M01 si M02 la mai putin de o ora dupa ce ai primit lotul. E cel mai scurt timp de reactie pe care l-ai avut vreodata.

Poarta de compilare: trece. `javac` pe ambele iese cu 0 erori. Le-am rulat de cate doua ori fiecare; tot ce scriu mai jos e reprodus prin rulare, nu dedus.

Ce e bine, si nu e putin:

- La **M01** ai pus `join()` intr-o bucla **separata** de `start()` (`M01.java:77` si `M01.java:80`). Asta era miezul proiectului, si multi il rateaza: cu ambele in aceeasi bucla ai fi pornit o pagina, ai fi asteptat-o, abia apoi ai fi pornit-o pe urmatoarea. Verdictul tau iese corect, `in paralel`, cu 1061 ms fata de o suma de 3832 ms.
- La **M02** ai prins exact cerinta grea: diferenta dintre cele doua variante e de un singur cuvint, `start()` la linia 50 si `run()` la linia 72. Numele firelor confirma: `Thread-0..3` la A, de patru ori `main` la B.

Problema rundei e una singura, si e la M02.

Critice

B1 ● Verdictul final spune exact pe dos fata de numerele pe care tocmai le-ai tiparit

`src/M02-cronometru-pacalit/M02.java:82`

CUM E ACUM: rulat acum, output integral la final:

```
Vaianta A : 531ms
Vaianta B : 2015ms
Vaianta A a fost de 0.2630272952853598ori mai rapid
```

Ai masurat corect: A face 531 ms, B face 2015 ms. Apoi programul anunta ca A a fost de **0.26** ori mai rapid — adica de aproape patru ori mai **lent**. Cele doua durate spun una, concluzia spune alta, in acelasi output.

DE CE: "de cate ori e A mai rapid decat B" inseamna *cate durate de A incap intr-o durata de B*. Deci A e impartitorul, nu impartitul. Tu ai scris `durataA/durataB`, adica exact fractia rasturnata.

Semnalul pe care il poti folosi singur data viitoare: la orice intrebare de forma "de cate ori mai rapid / mai mare / mai mult", un rezultat **subunitar** inseamna ca ai inversat fractia. `0.26` nu putea fi niciodata raspunsul corect la o intrebare care incepe cu "de cate ori mai rapid".

FIX:

acum	corect
<code>double n = (double)durataA/durataB;</code>	<code>double n = (double)durataB/durataA;</code>
tipareste <code>0.2630272952853598</code>	tipareste <code>3.8</code>

Pentru afisare, rotunjeste: `Math.round(durataB * 10.0 / durataA) / 10.0`.

Si un lucru de retinut dincolo de linia asta. Cerinta avea doua parti: "*tipareste «A a fost de N ori mai rapid»" si "cu N calculat"*". Ai facut-o pe prima si ai ratat-o pe a doua. E a patra oara cand se intampla exact asta: la recap ai

adus inapoi metoda `close()` dar n-ai chemat-o din `Main`, si ai promovat `inTransit` la camp dar nu l-ai incrementat nicaieri. De fiecare data, partea care se vede e facuta, partea care da inteles nu. Cand o cerinta are un "si", trateaz-o ca pe doua cerinte separate si bifeaza-le una cate una.

Importante

M1 🟡 La varianta B refolosesti fire care si-au trait viata

`src/M02-cronometru-pacalit/M02.java:72-75`

CUM E ACUM: `s1..s4` au fost pornite cu `start()` la liniile 50-53 si asteptate cu `join()` la 55-58. Cand ajungi la linia 72, cele patru fire sunt deja **terminated**. Tu chemi `s1.run()` pe ele.

DE CE: merge, dar din motivul gresit. `run()` e o metoda obisnuita, ca oricare alta — n-are nicio treaba cu starea firului, deci o poti chema de cate ori vrei, pe orice obiect, si se executa pe firul care o cheama. Daca in schimb ai fi scris `s1.start()` a doua oara, ai fi primit `IllegalThreadStateException`: **un Thread se porneste o singura data**.

Deci codul tau e corect din intamplare, nu din constructie. Un cititor care vede `s1.run()` dupa `s1.join()` nu stie daca ai vrut asta sau ai uitat ca obiectele sunt consumate.

FIX: construiesti patru obiecte noi pentru varianta B. Devine clar ca cele doua variante executa aceeasi *munca*, nu ca refolosesti aceleasi *fire*.

M2 🟡 A ramas o linie de depanare in mijlocul raportului

`src/M01-descarcator-pagini/M01.java:82`

```
finished produse
test          <- aici
acasa-760
```

`System.out.println("test")` sta intre bucla de `join()` si raport. Nu strica nimic, dar apare in livrare. Cauta-le inainte de commit.

Cleanups

C1 🟢 Ai scris getteri si apoi nu-i folosesti

`src/M01-descarcator-pagini/M01.java:91-93`

La linia 91 citesti prin `pagina.getNume()` si `pagina.getDurata()`, iar imediat dedesubt, la 92 si 93, ajungi direct la camp: `pagina.durata`. Aceeasi informatie, doua cai. Alege una — daca ai scris getterii, foloseste-i peste tot.

C2 🟢 Marunte la M02

- `Vaianta` in loc de `Varianta`, de trei ori (liniile 63, 79, 83).
 - `Medtoda` la linia 41.
 - Lipseste un spatiu: `+ n + "ori mai rapid "` produce `3.8ori`.
-

Intrebari de verificare

1. Ai tiparit `531ms` , apoi `2015ms` , apoi "de 0.26 ori mai rapid". Fara sa te uiti la cod: care dintre cele trei numere e gresit si de unde stii?
2. La varianta B ai chemat `s1.run()` pe un obiect pornit deja cu `start()` . Ce s-ar fi intamplat daca scriai `s1.start()` a doua oara, si de ce?
3. La M01, de ce `join()` trebuie sa fie in alta bucla decat `start()` ? Ce numar din raportul tau s-ar schimba daca le-ai pune in aceeasi bucla, si cum?

Urmatorul pas: M03, M04, M05. Hinturile sunt in `README.md` , dar citeste-le abia dupa ce te blochezi.